Full-Stack · SaaS · Realtime Systems

Eyad Syam

Building production SaaS products — from database design through real-time architecture, to polished user interfaces. No templates. No shortcuts.

Email eyadsyam124@gmail.com GitHub _____ Location Egypt · Remote

Availability 1 week notice

ENGINEERING APPROACH

Three principles that shape every build

These aren't buzzwords — they're the opinions I bring to every project. They're why my code survives audits, onboards new developers quickly, and doesn't need rewrites six months later.

01 · FOUNDATION

Security at the database layer

Row-Level Security policies enforced in Postgres, not just the UI. If an attacker bypasses the frontend, the database still refuses unauthorized queries. Every table ships with explicit policies — no "allow all" shortcuts.

02 · VELOCITY

Type safety end-to-end

TypeScript in strict mode, Zod schemas for runtime validation, database types generated from the real schema. Most bugs get caught at compile time — not by users in production at 2 AM.

03 · HANDOFF

Readable code, not clever code

Any mid-level developer should understand the codebase after one afternoon. I favor explicit over abstract, documented over intuitive, and boring tech over exciting tech. Predictability compounds.

RECENT ENGAGEMENT

By the numbers

Measurements from the TaskFlow build — a representative mid-complexity SaaS project.

8 tables DATABASE SCHEMA	**15** policies RLS RULES	**25K+** LOC TYPESCRIPT WRITTEN	**5** GB MAX FILE UPLOAD
30 fps UI ANIMATIONS	**<1.5** s FIRST CONTENTFUL PAINT	**0** errors TSC STRICT MODE	**RTL** + AR FULL LOCALIZATION

CASE STUDY · 2026

TaskFlow — Team collaboration SaaS

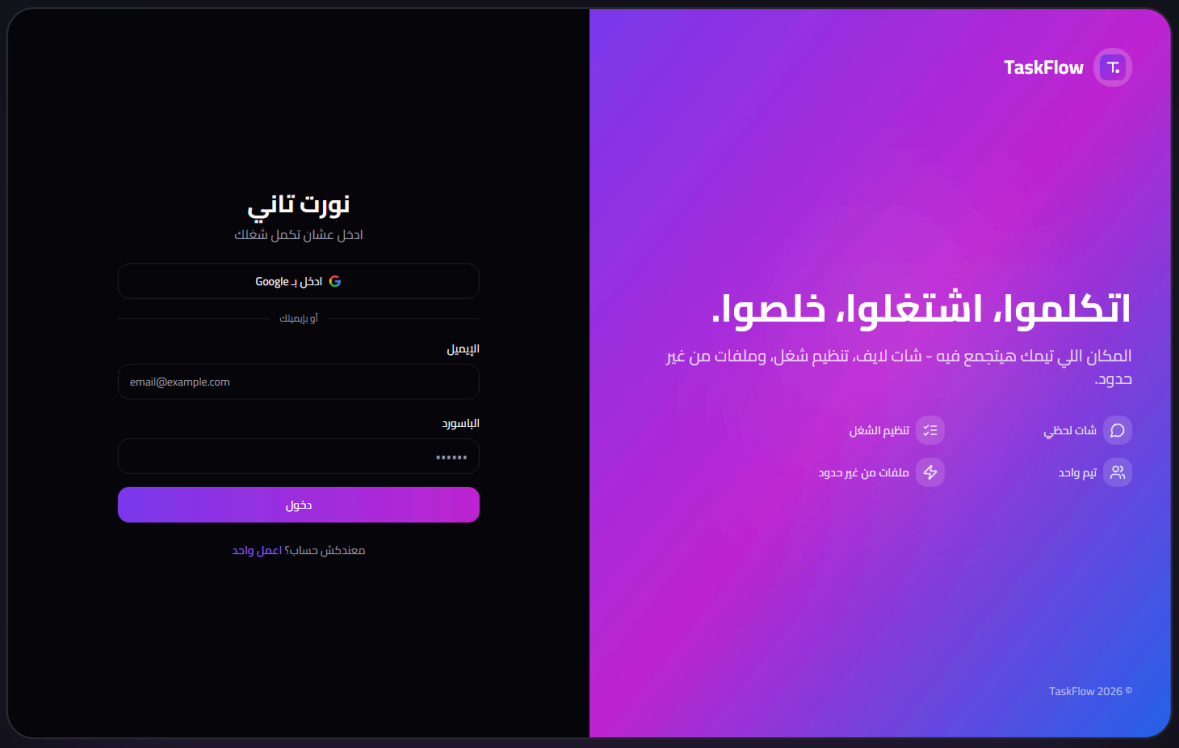
A complete team platform combining real-time chat, task management, and file sharing. Built ground-up with zero boilerplate, deployed to production, and handling the workflows of a real working team.

TaskFlow

LIVE IN PRODUCTION

Three integrated workflows: Real-time Chat · Task Management · File Sharing

Next.js 14 | TypeScript | Postgres 15 | Supabase | Realtime Broadcast | Tailwind CSS | Radix UI | React Hook Form | Zod | Arabic + RTL | Vercel Edge



The problem

Teams juggle 3+ tools to coordinate work: Slack for chat, Trello/Asana for tasks, Google Drive for files. Context switches kill focus, information scatters across apps, and there's no single source of truth for "what's everyone working on right now."

The solution

TaskFlow collapses these three workflows into one cohesive product designed for a single team (not multi-tenant). Every task has its own discussion room. Every team member's presence is visible from any screen. Every file is reachable from chat. Nothing lives in a separate tool.

> **Design choice:** Single-tenant architecture. Trading multi-tenancy for dramatically simpler data access, zero tenant-isolation bugs, and a faster product. When you're building for a specific team, not SaaS-for-SaaS-builders, this is the right call.

Core feature set

Real-time chat

Channel-based + direct messages. Live delivery under 200ms via Supabase Realtime broadcast. Typing indicators, emoji reactions, reply threads, message editing and deletion.

File uploads

5GB per file, no count limit. Direct browser-to-storage uploads with progress tracking. Inline preview for images, video, and audio. Paste-from-clipboard support.

Task workflow

Kanban and table views of the same data. Four-state workflow: pending client → in progress → awaiting payment → paid/closed. Database-level lock on paid tasks.

Live presence

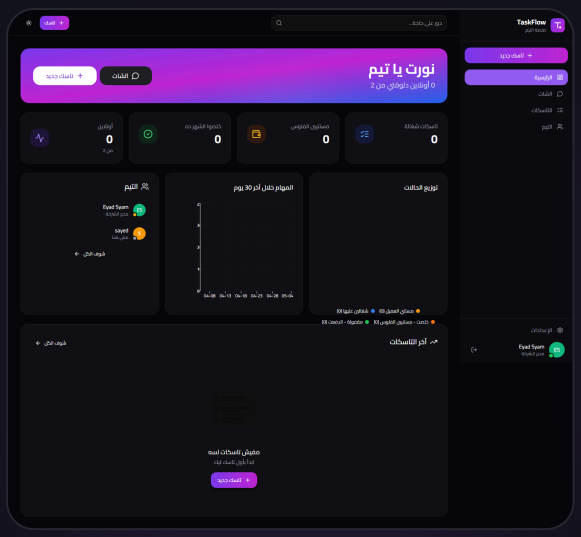
Every avatar shows online/away/offline status, updated in real-time across all connected clients. Status messages for "what I'm working on."

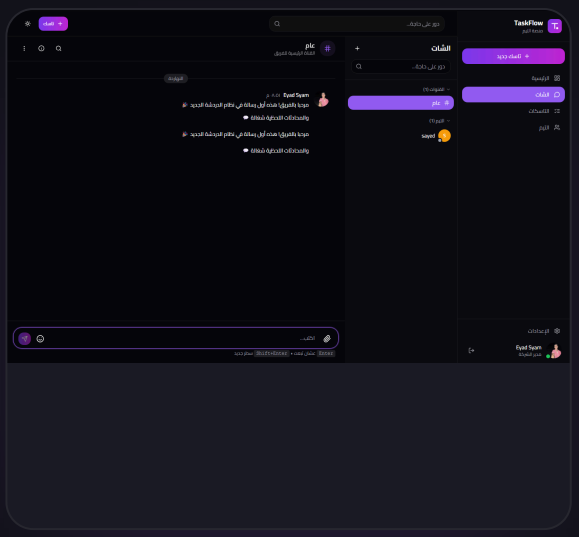
Onboarding wizard

Three-step personalization flow triggered for new users. Captures name, work info, status. Auto-enrolls into the team's default channel on completion.

Per-task chat rooms

Every task can spawn its own dedicated conversation. Links the business data (tasks) to the communication (chat) — discussions stay attached to the work.





Database architecture

Eight tables with proper foreign keys and indexed hot paths. Every table has explicit RLS policies — authorization is enforced at the database layer, not just the UI. This means even if an attacker bypasses the frontend entirely, the database still refuses unauthorized queries.

profiles	users + preferences
id PK	uuid
full_name	text
email	text
role	text
avatar_url	text
status_message	text
last_seen_at	timestamptz
onboarding_completed	boolean

tasks	work items
id PK	uuid
title	text
description	text
status	text
assigned_to FK	uuid
price	numeric
is_locked	boolean
tags	text[]

conversations	chat rooms
id PK	uuid
type	text
name	text
task_id FK	uuid
is_private	boolean
last_message_at IDX	timestamptz

messages	chat messages
id PK	uuid
conversation_id IDX	uuid
author_id FK	uuid
content	text
reply_to_id FK	uuid
is_edited	boolean

created_at IDX	timestamptz

message_attachments — files in messages

id PK	uuid
message_id FK	uuid
file_url	text
file_name	text
file_size	bigint
file_type	text

message_reactions — emoji reactions

id PK	uuid
message_id FK	uuid
user_id FK	uuid
emoji	text
UNIQUE(msg, user, emoji)	constraint

Representative RLS policy

Example of how authorization is enforced at the database level. This policy ensures a user can only update their own profile — even a compromised client with a leaked JWT can't modify other users.

```
CREATE POLICY "profiles_update_own" ON public.profiles
  FOR UPDATE
  USING (auth.uid() = id)
  WITH CHECK (auth.uid() = id);

-- Lock trigger: prevent editing paid_closed tasks from ANY source
CREATE OR REPLACE FUNCTION prevent_locked_task_edit()
RETURNS TRIGGER AS $$
BEGIN
  IF OLD.status = 'paid_closed' THEN
    RAISE EXCEPTION 'Cannot modify a paid and closed task (task_id=%)', OLD.id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

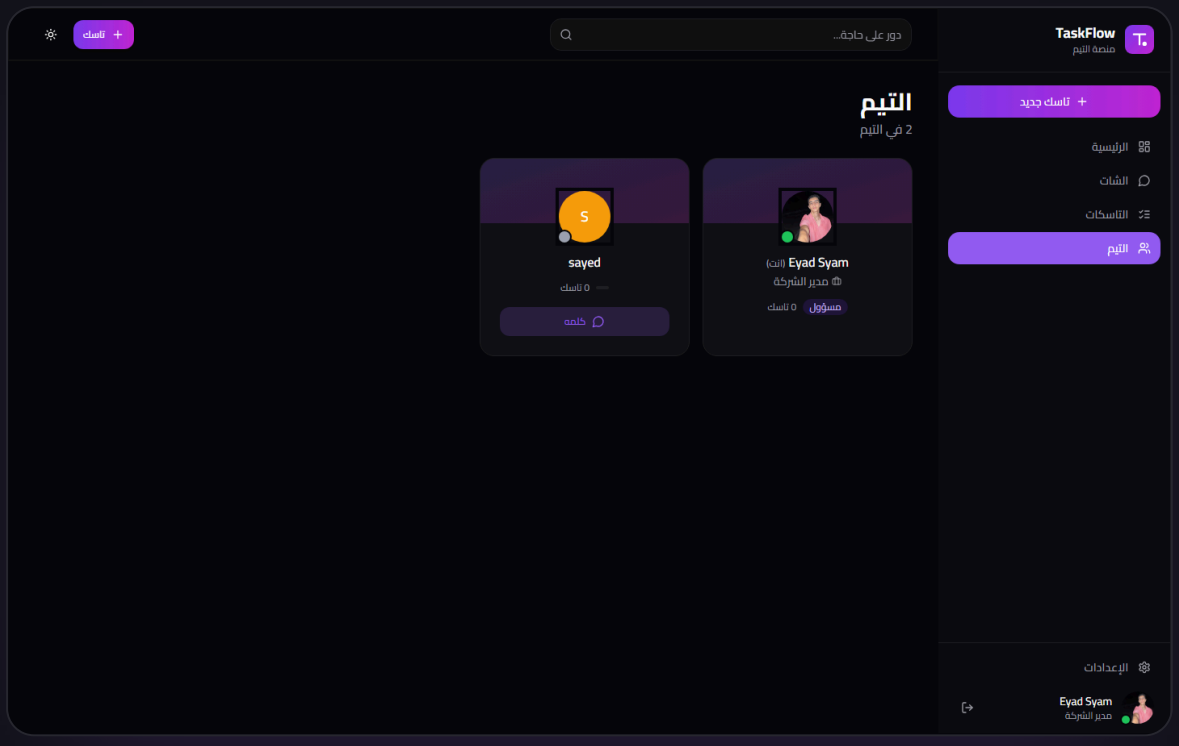
Real-time architecture

TaskFlow uses two distinct real-time channels for different use cases:

— **Postgres CDC (Change Data Capture)** — streams database changes directly to subscribed clients. Used for messages, reactions, and profile updates. Zero additional backend code needed.

- **Realtime Broadcast** — ephemeral pub/sub for events that don't need persistence. Used for typing indicators, which would otherwise pollute the database with short-lived rows.
- **Per-conversation scoping** — every subscription is filtered to a specific conversation_id. Prevents unnecessary network traffic from events the user isn't viewing.
- **Optimistic updates** — messages appear in the UI before server confirmation, then silently reconcile on roundtrip. Feels instant even on slow networks.

```
// lib/chat-helpers.ts — realtime subscription setup
const channel = supabase.channel(`conversation-${id}`)
  // Postgres CDC for persistent messages
  .on("postgres_changes", {
    event: "INSERT",
    schema: "public",
    table: "messages",
    filter: `conversation_id=eq.${id}`,
  }, handleNewMessage)
  // Broadcast for typing — doesn't hit the DB
  .on("broadcast", { event: "typing" }, handleTyping)
  .subscribe();
```



Middleware — the trickiest state machine

The auth middleware handles four distinct states on every request, with appropriate redirects for each. This logic caused 3 iterations during the build because the edge cases are subtle:

- **Unauthenticated user on protected route** → redirect to /login with `?next=` parameter preserving destination
- **Authenticated user without a profile row** → the trigger usually creates it, but for OAuth edge cases, middleware tolerates missing profiles and lets onboarding handle creation
- **Authenticated user who hasn't completed onboarding** → forced redirect to /onboarding from any page except auth callbacks (so OAuth flow completes before redirect)
- **Fully onboarded user on auth pages** → redirect to /dashboard (so refreshing /login doesn't show login form to logged-in users)

Custom design system

No UI library was used for the final look — Radix UI provides accessible primitives, but every visual component (button, input, card, dialog, popover, dropdown) was built custom to enable a coherent design language, dark mode by default, and full RTL support for Arabic layouts.

Typography scale

Cairo font family for Arabic, Inter for Latin. Letter-spacing negative at display sizes, positive at caption sizes. Feature flags enabled: `"rlig" 1, "calt" 1, "ss01" 1`.

Color tokens

HSL variables in globals.css for both light and dark themes. 40+ semantic tokens (primary, accent, destructive, sidebar, chat-bubble, status-*). Theme switch swaps the root attribute — no JS re-render.

Motion design

Slide-up, fade-in, scale-in animations for state transitions. Typing indicator uses staggered CSS keyframes. Pulse-ring effect for active states. All animations respect `prefers-reduced-motion`.

RTL-safe layouts

Physical props (`right-0`, `pr-72`) used deliberately for the sidebar to avoid RTL logical-property bugs. Text uses logical props. Every layout was manually tested in RTL mode.

Localization

Not just translation — the entire product was written for an Egyptian Arabic-speaking team. UI copy uses colloquial dialect ("يلا", "معندكش حساب", "نورت تاني", "نبدأ") instead of formal MSA. Date labels use colloquial forms ("النهاردة" for today, "إمبارح" for yesterday). Custom SVG illustrations replaced emoji decorations — for a more professional feel.

PROPOSED TECHNOLOGY

Stack I'd propose for your project

Each layer is chosen for a specific reason. The goal is rapid delivery today, sustainable maintenance tomorrow, and easy handoff to any developer who joins later. Open to alternatives based on your team's existing stack or preferences.

LAYER	PRIMARY CHOICE	REASONING
Framework	`Next.js 14 (App Router)`	Server components reduce client JS bundle. Built-in routing, image optimization, edge runtime.
Language	`TypeScript (strict)`	Compile-time type safety prevents a class of production bugs entirely.
Database	`Postgres (Supabase)`	Full SQL, proper relational model, RLS for authz, migrations, and realtime in one product.
ORM / Query	`supabase-js + raw SQL`	No ORM overhead. Type-safe generated types. SQL where performance matters.
Authentication	`Supabase Auth`	OAuth providers, JWT sessions, email flows — all configurable from dashboard.
Realtime	`Supabase Realtime`	Postgres CDC + broadcast in one service. Replaces need for custom WebSocket server.
File storage	`Supabase Storage`	S3-compatible, direct browser uploads, CDN-backed, RLS-protected by default.
UI primitives	`Radix UI`	Accessible headless components. Keyboard navigation and ARIA built in.

LAYER	PRIMARY CHOICE	REASONING
Styling	`Tailwind CSS`	Utility-first, consistent design tokens, easy dark mode, minimal runtime CSS.
Forms	`React Hook Form + Zod`	Uncontrolled inputs (fast), type-safe validation, inferred types from schemas.
Server state	`TanStack Query`	Caching, background refetching, optimistic updates, out-of-the-box.
Client state	`Zustand`	Simpler than Redux, faster than Context, handles non-server state cleanly.
Date handling	`date-fns`	Tree-shakeable, immutable, locale-aware. Avoids moment.js bloat.
Testing	`Playwright`	Real-browser E2E testing. Catches integration bugs unit tests miss.
Analytics	`Vercel Analytics / Plausible`	Privacy-friendly, no cookie banner required, lightweight.
Error tracking	`Sentry`	Session replay, source maps, performance monitoring — free tier covers most MVPs.
Email	`Resend`	Modern API, React Email templates, great deliverability.
Payments	`Stripe / Paymob`	Stripe for international, Paymob for Egypt-focused MENA payments.
Deployment	`Vercel`	Zero-config for Next.js, global edge network, instant rollbacks.
CI/CD	`GitHub Actions`	Typecheck + lint on PR, auto-deploy on merge, free for public repos.

PROCESS

How a 7-week build breaks down

Fixed phases with concrete deliverables at each milestone. You see working software by end of Week 2, not a blank screen for 6 weeks.

WEEK 1 · DISCOVERY

Deep-dive call to understand the business problem and users. Database schema design (ERD). API contract. Design direction (colors, typography, component style). Development environment provisioned.

DELIVERABLES

Technical spec document · Entity Relationship Diagram · Figma mockups or design direction document · GitHub repository with CI/CD configured · Supabase project provisioned

WEEK 2 · FOUNDATION

Database migrations written and applied. RLS policies authored with explicit tests. Authentication flow working (email + OAuth). First batch of UI components built. Staging environment deployed — clickable shell.

DELIVERABLES

Working auth on staging URL · 10+ UI component primitives · Database schema with RLS · CI pipeline enforcing typecheck and lint on every PR

WEEKS 3-4 · CORE FEATURES

Main CRUD flows for the core entity of your product. Dashboard and data visualization. Admin surfaces if needed. Real-time features (if applicable). File uploads (if applicable).

DELIVERABLES

All primary user flows functional on staging · Realtime infrastructure in place · File handling working end-to-end · Internal review document with known limitations

WEEK 5 · INTEGRATION

Payment integration (if applicable). Email notifications (if applicable). Webhook handling. Mobile-responsive tested on real devices. Error states designed and implemented.

DELIVERABLES

All integrations working · Mobile tested on iOS Safari + Android Chrome · Error boundary components in place · Loading states throughout

WEEK 6 · POLISH AND TESTING

Bundle size analysis and optimization. Database index review for hot queries. Cross-browser testing (Chrome, Safari, Firefox, Edge). Keyboard navigation audit. Playwright end-to-end tests for critical paths.

DELIVERABLES

Lighthouse score 90+ on key pages · E2E tests covering login → core workflow · Accessibility audit findings addressed · Performance baseline documented

WEEK 7 · LAUNCH

Production environment set up with custom domain and SSL. Monitoring and error tracking configured. Documentation written: README, deployment guide, environment variables reference, database migration guide. Video walkthrough recorded for future developers.

DELIVERABLES

Live production site · Documentation covering every environment variable, migration command, and deployment step · Loom video walkthrough of the codebase · 2 weeks of included post-launch bug-fix support begins

HOW I WORK

Working together

Operational details that matter more than they sound. How we communicate, how you stay informed, and how I avoid surprises.

A Async-first communication

I default to written communication over calls. Reduces interruptions and creates a searchable record. Calls for scoping, decisions, and kick-off — async for everything else.

B Daily progress updates

Every work day ends with a Loom recording or written summary: what shipped, what's blocked, what's tomorrow. You never wonder "what's the status?"

C GitHub from day one

You have access to the repository from the first commit. Every PR is reviewable. No hidden work. If you have senior developers on your team, they can review my code.

D Staging from week 2

A deployed, interactive staging URL is live from the second week. You click through real builds rather than reviewing mockups — catches misunderstandings early.

E Fixed-price milestones

Not hourly. I take the risk of estimate inaccuracy — you get cost certainty. Each milestone has a clear deliverable and payment tied to it.

F No lock-in

Standard tech, standard hosting, commented code. You can fire me tomorrow and any competent developer can continue. Your product does not depend on me.

INVESTMENT

Fixed-price tiers

These are indicative ranges based on common project shapes. A firm quote with milestone breakdown comes after a 30-minute scoping call. Fixed price per milestone, never hourly.

MVP

$2.5K – $4K

5–7 weeks · 2–3 milestones

- Authentication with 1 OAuth provider
- 1 primary user flow (CRUD + validation)
- Dashboard with 3–4 KPIs
- Basic admin surface for seed data
- Mobile responsive
- Deployed to production

MOST REQUESTED

Standard SaaS

$4K – $7.5K

7–10 weeks · 4–5 milestones

- Everything in MVP, plus:
- Multi-step onboarding flow
- 3–4 primary user flows
- Dashboard with charts and analytics
- Role-based access control
- Email notifications (transactional)
- Full admin panel with CRUD for all entities
- Search and filtering

Complex

$7.5K – $12K

10–14 weeks · 5–7 milestones

- Everything in Standard, plus:
- Real-time features (chat, presence, live data)
- External integrations (payments, APIs)
- Advanced file handling
- Webhook infrastructure
- Audit logging
- Advanced permissions / teams
- E2E test coverage for critical paths

What's included in every engagement

- ✓ Complete source code ownership — GitHub repository transferred to you at project end
- ✓ Database schema, all migrations, and a documented rollback procedure
- ✓ Deployment to your preferred host (Vercel, AWS, Hetzner, or self-hosted)
- ✓ Technical documentation covering architecture, environment variables, and deployment
- ✓ Loom video walkthrough of the codebase for your future developers
- ✓ Two weeks of post-launch bug-fix support (included)
- ✓ Staging environment accessible from Week 2 onwards
- ✓ GitHub-based workflow — every commit is visible, every PR is reviewable

Billed separately (if needed)

- — Ongoing feature work after the 2-week post-launch support window
- — Custom design work if you don't have a visual direction (I can recommend a designer)
- — Third-party service fees — Supabase, Vercel, domain, email provider, Stripe fees (you own these accounts, typical MVP cost: $0–$50/month)
- — Content writing — copy, onboarding text, email templates (I can draft, you approve)
- — Migrations from legacy systems (quoted per-project based on data complexity)

CODE QUALITY

You'll judge the code, not the pitch

Everything above is claims. The best way to evaluate is to read the code. The TaskFlow repository is public — walk through it with a senior developer if you want to verify my technical claims before committing.

Zero TypeScript errors in strict mode

The project compiles with `tsc --noEmit` cleanly. No `// @ts-ignore` or `as any` shortcuts hiding bugs.

No ESLint warnings

Lint runs in CI on every PR. Warnings are treated as errors for anything non-stylistic.

Readable database migrations

Each migration is a separate, named file with clear intent. Rollback procedures documented. No ad-hoc schema changes.

Meaningful commit messages

Commit history tells the story of the product. Every commit has context for why the change was made, not just what changed.

Bottom line: If you have a technical co-founder or CTO, hand them the GitHub URL and ask for a 20-minute review. If they say "this is fine", you've just validated my technical claims for free.

Ready when you are

A 30-minute call is enough to understand your project, walk you through the TaskFlow codebase, and send a firm quote with milestone breakdown. No obligation.

Email me to schedule a call

© 2026 Eyad Syam · eyadsyam124@gmail.com · github.com/eyadsyam

This portfolio is a static HTML document generated from the TaskFlow project. Source: /proposal/portfolio.html